

Worksheet -4



Name: BINA THAPA

Module Title: Programming in C++

Student id- 23085129

Task 1: Solve the following programming problems:

1. STL Container Practice: Write a program using STL containers that:

1. Uses vector<string> to store names
2. Uses map<string, int> to store age against each name
3. Implements functions to:
 1. Add new name-age pair
 2. Find all people above certain age
 3. Sort and display names alphabetically

➤ Answer:

```
#include <iostream>
#include <vector>
#include <map>
#include <algorithm>

using namespace std;

class Alpha
{
protected:
    vector <string> names;
    map <string,int> ages;
public:
    void addname(string name,int age)
    {
        names.push_back(name);
        ages[name]=age;
        cout<<"New name: "<<name<<" and age: "<<age<<" years" <<" is
added"<<endl;
    }
    void above(int aboveage)
    {
        cout<<"Names above the age of "<<aboveage<<" are: "<<endl;
        bool a=false;
        for(auto naam : ages )
        {
            if (naam.second> aboveage)
            {
                cout<<naam.first <<" " <<naam.second <<" "<<"years"<<endl;
                a=true;
            }
        }
        if (!a)
```

```

        {
            cout<<"Everyone is under"<<aboveage<<endl;
        }
    }
    void display()
    {
        cout<<"Names in alphabetical order: "<<endl;
        vector<string> newnames = names;
        sort(newnames.begin(),newnames.end());
        for (int i=0;i<newnames.size();i++)
        {
            cout<< newnames[i] << "-"<<ages[newnames[i]]<<endl;
        }
    }

};

int main()
{
    Alpha a1;
    int ch;
    do
    {
        cout<<" ----- Menu -----"<<endl;
        cout<<" 1. Add new names "<<endl;
        cout<<" 2. Check the age limit "<<endl;
        cout<<" 3. Display Names Alphabetically "<<endl;
        cout<<" 4. Exit "<<endl;
        cout<<"Enter your choice: "<<endl;
        cin>>ch;

        if(ch==1)
        {
            string name;
            int age;
            cout << "Enter name: ";
            cin >> name;
            cout << "Enter age: ";
            cin >> age;

```

```

        a1.addname(name,age);

    }

    else if(ch==2)
    {
        int Limit;
        cout << "Enter age limit: ";
        cin >> Limit;
        a1.above(Limit);

    }

    else if (ch==3)
    {
        a1.display();

    }

    else if (ch==4)
    {
        cout<<"----- The end -----"<<endl;
        break;
    }

    else
    {
        cout<<"Invalid choice!!"<<endl;
        break;
    }
    }while(ch !=4);
return 0;
}

```

❖ Output:



"C:\Users\beena\OneDrive\De" X



----- Menu -----

1. Add new names
2. Check the age limit
3. Display Names Alphabetically
4. Exit

Enter your choice:

1

Enter name: Bina

Enter age: 21

New name: Bina and age: 21 years is added

----- Menu -----

1. Add new names
2. Check the age limit
3. Display Names Alphabetically
4. Exit

Enter your choice:

1

Enter name: Sharad

Enter age: 21

New name: Sharad and age: 21 years is added

----- Menu -----

1. Add new names
2. Check the age limit
3. Display Names Alphabetically
4. Exit

Enter your choice:

1

Enter name: Bishwas

Enter age: 15

New name: Bishwas and age: 15 years is added

```
----- Menu -----
1. Add new names
2. Check the age limit
3. Display Names Alphabetically
4. Exit
Enter your choice:
1
Enter name: Lina
Enter age: 27
New name: Lina and age: 27 years is added
----- Menu -----
1. Add new names
2. Check the age limit
3. Display Names Alphabetically
4. Exit
Enter your choice:
2
Enter age limit: 18
Names above the age of 18 are:
Bina 21 years
Lina 27 years
Sharad 21 years
```

```
----- Menu -----
1. Add new names
2. Check the age limit
3. Display Names Alphabetically
4. Exit
Enter your choice:
3
Names in alphabetical order:
Bina-21
Bishwas-15
Lina-27
Sharad-21
----- Menu -----
1. Add new names
2. Check the age limit
3. Display Names Alphabetically
4. Exit
Enter your choice:
4
----- The end -----

Process returned 0 (0x0)    execution time : 72.568 s
Press any key to continue.
|
```

2. Stack Problem: Implement a stack using arrays (not STL) that:

1. Has basic push and pop operations
2. Has a function to find middle element
3. Has a function to reverse only bottom half of stack
4. Maintain stack size of 10

➤ Answer:

```
#include <iostream>
using namespace std;
```

```
class Stackarray
```

```
{
protected:
```

```
    int array[10];
```

```
    int a;
```

```
    int value;
```

```
    int middle;
```

```
    int s;
```

```
public:
```

```
    Stackarray()
```

```
    {
        a=-1;
```

```
    }
```

```
    void push()
```

```
    {
```

```
        if (a==9)
```

```
        {
```

```
            cout<<"Stack is full!"<<endl;
```

```
        }
```

```
        cout<<"Enter new value: "<<endl;
```

```
        cin>>value;
```

```
        array[++a]=value;
```

```
        cout<<value<<" is pushed to stack "<<endl;
```

```
    }
```

```
    void pop()
```

```
    {
```

```

    if(a== -1)
    {
        cout<<"No elements yet in stack"<<endl;

    }
    cout<< array[a--]<<" is popped from stack"<<endl;
}
void middlenum()
{
    if (a== -1)
    {
        cout<<"No elements yet in stack"<<endl;

    }
    middle= a/2;
    cout<<array[middle]<<" is at the middle of stack"<<endl;

}
void reverse()
{
    if(a<1)
    {
        cout<<" Not enough elements!!"<<endl;
    }
    int start = 0;
    int end = (a / 2);
    while (start < end)
    {
        s = array[start];
        array[start] = array[end];
        array[end] = s;
        start++;
        end--;
    }

    cout << "Bottom half of stack is reversed"<<endl;

}

void display()
{
    if (a== -1)

```



```

    {
        cout<<"No elements yet in stack"<<endl;

    }
    cout<<" Elements from top to bottom"<<endl;
    for (int i = a; i >= 0; i--)
    {
        cout << array[i] << endl;
    }
    cout<<endl;

}
};

```

```

int main()
{
    Stackarray s1;
    int ch;

    do
    {
        cout<<" ----- Menu -----"<<endl;
        cout<<" 1.Push "<<endl;
        cout<<" 2.Pop"<<endl;
        cout<<" 3.find the middle "<<endl;
        cout<<" 4.Reverse the bottom half "<<endl;
        cout<<" 5.Display "<<endl;
        cout<<" 6. exit "<<endl;
        cout<<"Enter your choice: "<<endl;
        cin>>ch;

        switch(ch)
        {
            case 1:
                s1.push();
                break;
            case 2:
                s1.pop();
                break;
            case 3:
                s1.middlenum();
                break;
            case 4:

```

```
        s1.reverse();
        break;
    case 5:
        s1.display();
        break;
    case 6:
        cout<<"-----The End-----"<<endl;
        break;
    default:
        cout<<"Invalid input!!"<<endl;
        break;
}

}while(ch!=6);
return 0;

}
```

❖ Output

```
----- Menu -----
1.Push
2.Pop
3.find the middle
4.Reverse the bottom half
5.Display
6. exit
Enter your choice:
1
Enter new value:
10
10 is pushed to stack
----- Menu -----
1.Push
2.Pop
3.find the middle
4.Reverse the bottom half
5.Display
6. exit
Enter your choice:
1
Enter new value:
20
20 is pushed to stack
----- Menu -----
```

```
1.Push
2.Pop
3.find the middle
4.Reverse the bottom half
5.Display
6. exit
Enter your choice:
1
Enter new value:
30
30 is pushed to stack
----- Menu -----
1.Push
2.Pop
3.find the middle
4.Reverse the bottom half
5.Display
6. exit
Enter your choice:
1
Enter new value:
40
40 is pushed to stack
----- Menu -----
```

```

----- Menu -----
1.Push
2.Pop
3.find the middle
4.Reverse the bottom half
5.Display
6. exit
Enter your choice:
4
Bottom half of stack is reversed
----- Menu -----
1.Push
2.Pop
3.find the middle
4.Reverse the bottom half
5.Display
6. exit
Enter your choice:
5
Elements from top to bottom
40
30
10
20

```

```

2.Pop
3.find the middle
4.Reverse the bottom half
5.Display
6. exit
Enter your choice:
3
10 is at the middle of stack
----- Menu -----
1.Push
2.Pop
3.find the middle
4.Reverse the bottom half
5.Display
6. exit
Enter your choice:
2
40 is popped from stack
----- Menu -----
1.Push
2.Pop
3.find the middle
4.Reverse the bottom half
5.Display
6. exit
Enter your choice:
5
Elements from top to bottom
30
10
20

```

3. Queue Problem: Implement a queue using arrays (not STL) that:

1. Has basic enqueue and dequeue operations
2. Has a function to reverse first K elements
3. Has a function to interleave first half with second half
4. Handle queue overflow/underflow

➤ Answer:

```
#include <iostream>
using namespace std;
class Queue
{
protected:
    int array[10];
    int first;
    int last;
    int middle;
    int half[10];
    int n;
public:
    Queue()
    {
        first=-1;
        last=-1;
    }
    void enqueue(int value)
    {
        if (last==9)
        {
            cout<<"The Queue is full!"<<endl;
        }
        if(first== -1)
        {
            first=0;
        }
        array[++last]=value;
        cout<<value<<" Enqueued"<<endl;
    }
};
```

```

}
int dequeue()
{
    if(first==-1 || first>last)
    {
        cout<<"Queue is empty!"<<endl;

    }
    return array[first++];
}
void display()
{
    if(first==-1 || first>last)
    {
        cout<<"Queue is empty!"<<endl;
    }
    cout<<" Queue"<<endl;
    for(int i=first;i<=last;i++)
    {
        cout<<array[i]<<endl;
    }
    cout<<endl;
}
void reversek(int k)
{
    if(first==-1 || first>last)
    {
        cout<<"Queue is empty!"<<endl;
    }

    n = last - first + 1;
    if (k <= 0 || k > n)
    {
        cout << "Invalid value!!." << endl;
    }

    int stack[10], top = -1;

    for (int i = 0; i < k; i++)
    {
        stack[++top] = array[first + i];
    }
}

```

```

        for (int i = 0; i < k; i++)
        {
            array[first + i] = stack[top--];
        }

        cout << "First " << k << " elements reversed." << endl;
    }
void interleave()
{
    if(first== -1 || first>last)
    {
        cout<<"Queue is empty!"<<endl;
    }

    n = last - first + 1;
    middle=n/2;
    if (n%2!=0)
    {
        cout<<"Queue size should be even to interleave"<<endl;
    }
    for (int i = 0; i < middle; i++)
    {
        half[i] = dequeue();
    }

    for (int i = 0; i < middle; i++)
    {
        enqueue(half[i]);
        enqueue(dequeue());
    }

    cout << "Interleaving success"<<endl;
}
};

```

```

int main()
{
    Queue q;
    int ch, value, k;

    do {
        cout<<" ----- Menu -----"<<endl;

```

```
cout<<" 1.Enqueue "<<endl;
cout<<" 2.Dequeue"<<endl;
cout<<" 3.Display"<<endl;
cout<<" 4.Reverse the first k elemets "<<endl;
cout<<" 5.Interleave queue"<<endl;
cout<<" 6. exit "<<endl;
cout<<"Enter your choice: "<<endl;
cin>>ch;
```

```
switch (ch)
{
case 1:
    cout << "Enter value to enqueue: ";
    cin >> value;
    q.enqueue(value);
    break;

case 2:
    q.dequeue();
    break;

case 3:
    q.display();
    break;

case 4:
    cout << "Enter value of K: ";
    cin >> k;
    q.reversek(k);
    break;

case 5:
    q.interleave();
    break;

case 6:
    cout << "-----Exit-----"<<endl;
    break;

default:
    cout << "Invalid!!"<<endl;
}
```



```
    } while (ch!= 6);  
  
    return 0;  
}
```

❖ Output

```
----- Menu -----  
1.Enqueue  
2.Dequeue  
3.Display  
4.Reverse the first k elemets  
5.Interleave queue  
6. exit  
Enter your choice:  
1  
Enter value to enqueue: 10  
10 Enqueued  
----- Menu -----  
1.Enqueue  
2.Dequeue  
3.Display  
4.Reverse the first k elemets  
5.Interleave queue  
6. exit  
Enter your choice:  
1  
Enter value to enqueue: 20  
20 Enqueued  
----- Menu -----  
1.Enqueue  
2.Dequeue  
3.Display  
4.Reverse the first k elemets  
5.Interleave queue  
6. exit  
Enter your choice:  
1  
Enter value to enqueue: 30  
30 Enqueued
```

```
----- Menu -----
1.Enqueue
2.Dequeue
3.Display
4.Reverse the first k elemets
5.Interleave queue
6. exit
Enter your choice:
1
Enter value to enqueue: 40
40 Enqueued
----- Menu -----
1.Enqueue
2.Dequeue
3.Display
4.Reverse the first k elemets
5.Interleave queue
6. exit
Enter your choice:
1
Enter value to enqueue: 50
50 Enqueued
```

```
----- Menu -----
1.Enqueue
2.Dequeue
3.Display
4.Reverse the first k elemets
5.Interleave queue
6. exit
Enter your choice:
3
Queue
10
20
30
40
50
```

```

----- Menu -----
1.Enqueue
2.Dequeue
3.Display
4.Reverse the first k elemets
5.Interleave queue
6. exit
Enter your choice:
2
----- Menu -----
1.Enqueue
2.Dequeue
3.Display
4.Reverse the first k elemets
5.Interleave queue
6. exit
Enter your choice:
3
Queue
20
30
40
50

----- Menu -----
----- Menu -----
1.Enqueue
2.Dequeue
3.Display
4.Reverse the first k elemets
5.Interleave queue
6. exit
Enter your choice:
4
Enter value of K: 2
First 2 elements reversed.
----- Menu -----
1.Enqueue
2.Dequeue
3.Display
4.Reverse the first k elemets
5.Interleave queue
6. exit
Enter your choice:
3
Queue
30
20
40
50

```

```
----- Menu -----
1.Enqueue
2.Dequeue
3.Display
4.Reverse the first k elemets
5.Interleave queue
6. exit
Enter your choice:
5
30 Enqueued
40 Enqueued
20 Enqueued
50 Enqueued
Interleaving success
----- Menu -----
1.Enqueue
2.Dequeue
3.Display
4.Reverse the first k elemets
5.Interleave queue
6. exit
Enter your choice:
3
Queue
30
40
20
50
```

4.Linked List Problem: Create a singly linked list (not STL) that:

5. Has functions to insert at start/end/position
6. Has a function to detect and remove loops
7. Has a function to find nth node from end
8. Has a function to reverse list in groups of K nodes

➤ Answer:

```
#include <iostream>
using namespace std;
```

```
class Node
{
public:
    int data;
    Node* pointer;

    Node(int val)
    {
        data = val;
        pointer = nullptr;
    }
};
```

```
class List
{
private:
    Node* start;
    Node* tail;
```

```
public:
    List()
    {
        start = nullptr;
        tail = nullptr;
    }
```

```
void addFirst(int val)
{
    Node* newNode = new Node(val);
    if (start == nullptr)
    {
        start = tail = newNode;
    } else
```

```

    {
        newNode->pointer = start;
        start = newNode;
    }
}

void addLast(int val) {
    Node* newNode = new Node(val);
    if (start == nullptr) {
        start = tail = newNode;
    } else {
        tail->pointer = newNode;
        tail = newNode;
    }
}

void addAtPosition(int val, int pos)
{
    if (pos < 0)
    {
        cout << "Invalid position!" << endl;

    }
    if (pos == 0)
    {
        addFirst(val);

    }

    Node* temp = start;
    for (int i = 0; i < pos - 1; i++) {
        if (temp == nullptr) {
            cout << "Invalid position" << endl;
            return;
        }
        temp = temp->pointer;
    }

    Node* newNode = new Node(val);
    newNode->pointer = temp->pointer;
    temp->pointer = newNode;
}

```

```

void showList()
{
    Node* temp = start;
    while (temp != nullptr)
    {
        cout << temp->data << " -> ";
        temp = temp->pointer;
    }
    cout << "NULL" << endl;
}

```

```

void detectAndBreakLoop()
{
    Node* slow = start;
    Node* fast = start;
    while (fast && fast->pointer)
    {
        slow = slow->pointer;
        fast = fast->pointer->pointer;
        if (slow == fast)
        {
            breakLoop(slow);
            cout << "Loop detected and removed." << endl;
            return;
        }
    }
    cout << "No loop found." << endl;
}

```

```

void breakLoop(Node* loopNode)
{
    Node* ptr1 = start;
    Node* ptr2 = loopNode;
    while (ptr1->pointer != ptr2->pointer)
    {
        ptr1 = ptr1->pointer;
        ptr2 = ptr2->pointer;
    }
    ptr2->pointer = nullptr;
}

```

```

void findFromEnd(int n)
{

```

```

Node* mainPtr = start;
Node* refPtr = start;

for (int i = 0; i < n; i++)
{
    if (!refPtr)
    {
        cout << "Less than " << n << " elements" << endl;
        return;
    }
    refPtr = refPtr->pointer;
}

while (refPtr)
{
    mainPtr = mainPtr->pointer;
    refPtr = refPtr->pointer;
}

cout << n << "th element from end is: " << mainPtr->data << endl;
}

Node* Reverse(Node* node, int k)
{
    Node* prev = nullptr;
    Node* curr = node;
    Node* next = nullptr;

    Node* check = node;
    for (int i = 0; i < k; ++i)
    {
        if (!check) return node;
        check = check->pointer;
    }

    int count = 0;
    while (curr && count < k)
    {
        next = curr->pointer;
        curr->pointer = prev;
        prev = curr;
        curr = next;
        count++;
    }
}

```



```

    }

    if (next) node->pointer = Reverse(next, k);
    return prev;
}

void reverseGroups(int k)
{
    start = Reverse(start, k);
    cout << "List reversed in groups of " << k << ":" << endl;
    showList();
}
};

```

```

int main()
{
    List myList;
    int ch, num, pos, groupSize;

    do
    {
        cout << "\n-----Menu-----" << endl;
        cout << "1. Insert at End" << endl;
        cout << "2. Insert at Start" << endl;
        cout << "3. Insert at Position" << endl;
        cout << "4. Display List" << endl;
        cout << "5. Find Nth Node from End" << endl;
        cout << "6. Reverse List in Groups of K" << endl;
        cout << "7. Detect and Remove Loop" << endl;
        cout << "8. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> ch;
        switch (ch) {
            case 1:
                cout << "Enter value: ";
                cin >> num;
                myList.addLast(num); // Corrected logic
                break;
            case 2:
                cout << "Enter value: ";
                cin >> num;
                myList.addFirst(num);
                break;

```

```

        case 3:
            cout << "Enter value and position: ";
            cin >> num >> pos;
            myList.addAtPosition(num, pos);
            break;
        case 4:
            myList.showList();
            break;
        case 5:
            cout << "Enter n: ";
            cin >> pos;
            myList.findFromEnd(pos);
            break;
        case 6:
            cout << "Enter K: ";
            cin >> groupSize;
            myList.reverseGroups(groupSize);
            break;
        case 7:
            myList.detectAndBreakLoop();
            break;
        case 8:
            cout << "-----Exit-----\n";
            break;
        default:
            cout << "Invalid choice!!" << endl;
    }
} while (ch != 8);

return 0;
}

```

❖ Output



"C:\Users\beena\OneDrive\De" X



-----Menu-----

1. Insert at End
2. Insert at Start
3. Insert at Position
4. Display List
5. Find Nth Node from End
6. Reverse List in Groups of K
7. Detect and Remove Loop
8. Exit

Enter your choice: 1

Enter value: 4

-----Menu-----

1. Insert at End
2. Insert at Start
3. Insert at Position
4. Display List
5. Find Nth Node from End
6. Reverse List in Groups of K
7. Detect and Remove Loop
8. Exit

Enter your choice: 1

Enter value: 6

"C:\Users\beena\OneDrive\De" X + v

-----Menu-----

1. Insert at End
2. Insert at Start
3. Insert at Position
4. Display List
5. Find Nth Node from End
6. Reverse List in Groups of K
7. Detect and Remove Loop
8. Exit

Enter your choice: 1

Enter value: 5

-----Menu-----

1. Insert at End
2. Insert at Start
3. Insert at Position
4. Display List
5. Find Nth Node from End
6. Reverse List in Groups of K
7. Detect and Remove Loop
8. Exit

Enter your choice: 1

Enter value: 9

-----Menu-----

1. Insert at End
2. Insert at Start
3. Insert at Position
4. Display List
5. Find Nth Node from End
6. Reverse List in Groups of K
7. Detect and Remove Loop
8. Exit

Enter your choice: 2

Enter value: 6

-----Menu-----

1. Insert at End
2. Insert at Start
3. Insert at Position
4. Display List
5. Find Nth Node from End
6. Reverse List in Groups of K
7. Detect and Remove Loop
8. Exit

Enter your choice: 3

Enter value and position: 4 2

-----Menu-----

1. Insert at End
2. Insert at Start
3. Insert at Position
4. Display List
5. Find Nth Node from End
6. Reverse List in Groups of K
7. Detect and Remove Loop
8. Exit

Enter your choice: 4

6 -> 4 -> 4 -> 6 -> 5 -> 9 -> NULL

-----Menu-----

1. Insert at End
2. Insert at Start
3. Insert at Position
4. Display List
5. Find Nth Node from End
6. Reverse List in Groups of K
7. Detect and Remove Loop
8. Exit

Enter your choice: 5

Enter n: 3

3th element from end is: 6

```
-----Menu-----
1. Insert at End
2. Insert at Start
3. Insert at Position
4. Display List
5. Find Nth Node from End
6. Reverse List in Groups of K
7. Detect and Remove Loop
8. Exit
Enter your choice: 6
Enter K: 3
List reversed in groups of 3:
4 -> 4 -> 6 -> 9 -> 5 -> 6 -> NULL
-----Menu-----
```

```
-----Menu-----
1. Insert at End
2. Insert at Start
3. Insert at Position
4. Display List
5. Find Nth Node from End
6. Reverse List in Groups of K
7. Detect and Remove Loop
8. Exit
Enter your choice: 7
No loop found.
```